Ray Tracing
Geavanceerde computerarchitectuur

(B-KUL-JPI30R)

Elias Deruytter

December 16, 2023





semester: 1e semester

klas: MElecfs

Instructor: Marinšek Alex

1 Introduction

In this project, the primary objective was to develop a basic ray tracer utilizing CUDA, targeting the potential speed benefits offered by GPU use. The ray tracer aimed to simulate the interaction of light with surfaces in a 3D environment. The task involved creating an image by tracing rays originating from the camera and interacting with spheres within the scene. To make things more interesting, moving a sphere from A to B was implemented, this resulted in longer calculation times.

2 Ray tracing - goal

Ray tracing is a rendering technique used in computer graphics and visual effects to simulate the way light interacts with virtual objects in a three-dimensional environment. This method traces the path of rays of light as they travel through a scene, accurately modeling the behavior of light to generate realistic images.

With a digital camera, light from the light source reflects from objects and goes to the camera sensor. In ray tracing the opposite happens. rays (light rays) are created at the camera to reflect on spheres and eventually get back to the light source.

In what follows, a basic method is implemented. More advanced features of a ray tracer could be: shadow, reflection, changing light source,...

2.1 Camera - plane

First of all, the ray tracer consists of a fading background filled with chosen spheres. The system uses rays for every pixel that is wanted. A ray can actually be seen as a vector with a origin and a direction, as can be seen in figure 1. In other words a vector/ray can be made with 2 coordinates, one for the origin and one for the direction.

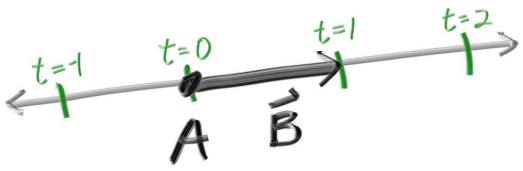


Figure 1: Representation ray

The next thing to think about is the camera from which rays are shot in combi-

nation with a plane to project the results on. A simple camera would be at the origin of the coordinate system. The plane can have for example 800*400 pixels and should be oriented perpendicular to the ray that intersects the plane in the center. It can also be seen as a small window in which light from the camera reaches the outdoor world. An illustration of this can be seen in figure 2

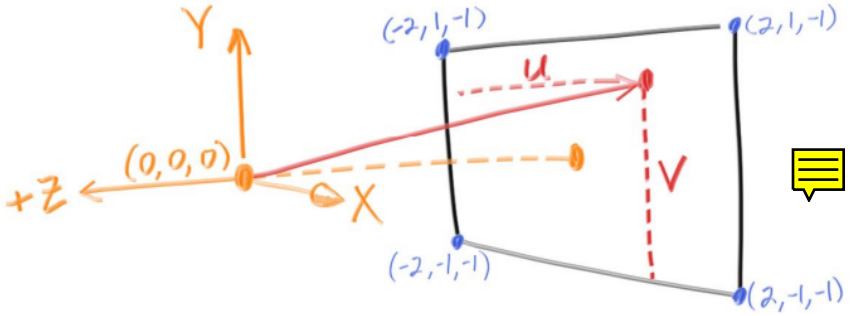


Figure 2: Camera and plane

2.2 spheres

A simple object for in the virtual world is a sphere. A sphere consists of a radius, a color and a position. If a ray intercepts a sphere, it typically intercepts it twice, once on the front side, once on the back side of the sphere, resulting in two answers. These answers can also be found mathematically. A simple formula for a sphere is this: $x^2 + y^2 + z^2 = R^2$, adding coordinates to the formula results in: $(x - cx)^2 + (y - cy)^2 + (z - cz)^2 = R^2$, with cx, cy and cz, the coordinates of the center of the sphere.

Another representation of this formula can be done with vectors (x,y,z vectors). $(x-cx)^2+(y-cy)^2+(z-cz)^2$ is actually equal to the dot product of p(x,y,z) and c(cx,cy,cz). So the equation of a sphere in vector form is now: $\text{dot}((p-c),(p-c)) = R^2$.

To find an intersection point, the p coordinates should be on the ray vector. This vector was defined with a origin A and a direction B and can be lengthened by multiplying the direction vector with 't'. this gives: $\text{dot}((A + B * t - c), (A + B * t - c)) = R^2$

Basic vector algebra for the dot product gives a new quadratic equation to calculate t:

$$t^2 \cdot \text{dot}(B, B) + 2t \cdot \text{dot}(B, A - C) + \text{dot}(A - C, A - C) - R^2 = 0$$

Where t represents the distance to the spherical surface. When t is known, and is bigger then 0, a sphere has been hit at coordinates $A + B * t$.

This formula can be easily used in code to calculate t, as is discussed in implementation (3).

To make the calculation more difficult, resulting in a longer and better timing, the sphere has also been moved from point A to point B in 30 frames to make a video.

3 Implementation

3.1 main.cu

To start off, some variables are declared. The width and height of the picture, this defines the amount of rays or the amount of threads that have to be executed, each thread has to process 1 ray. The amount of iterations is chosen to be 30, meaning the sphere changes location 30 times. A list of spheres is also created. As you can see in listing 1, the location, radius and color have to be given with the function call sphere(). A location or color always has 3 values, this is why it's called a vec3 object. Vec3 is actually a class for vectors [1], this class can do basic calculations on the 3 dimensional vectors (such as dot product). The window the camera is looking at is also defined here, this window has nx*ny pixels, each has to be given a color. The rgb values are stored in filevalues[nx*ny*3].

Listing 1: main.cu

```
int nx = 800; //width
int ny = 400; //height
int iterations = 30; //amount of frames in the video

int fileValues[nx*ny*3]; //3 for rgb
int length_list = 3; //amount of spheres
sphere list[length_list];
list[0] = sphere(vec3(0,100000,-1),99985,vec3(0.1,0.1,0.1));
   //ground - gray
list[1] = sphere(vec3(0,-2,-60),7,vec3(0,1,0)); //green
list[2] = sphere(vec3(20,-1,-80),5,vec3(1,0,0)); //red

//window camera looks at:
vec3 lowerleftcorner(-2.0,-1.0,-5);
vec3 horizontal(4.0,0.0,0.0);
vec3 vertical(0.0,2.0,0.0);
vec3 origin(0.0,0.0,0.0);
```

Next, memory has to be allocated in gpu. This is done with cudaMalloc(). It also has to be freed at the end.

```
int * d_fileValues;
int sizeFileValues = sizeof(fileValues);
sphere *d_list;

cudaMalloc((void **)&d_fileValues, sizeFileValues);
cudaMalloc((void**)&d_list, sizeof(sphere) * length_list);
```

Now every iteration, a global function 'calculateColor()' is called. But before that, every iteration needs it's own file name (e.g. output3.ppm) and a list of spheres. In this case 1 sphere changes position every iteration. Ppm (Portable Pix Map format) files where chosen because these files only require a simple header to be followed with the pixel data. When the global function is finished the data from d_filevalues is copied from gpu to filevalues in cpu. At the end the file itself is created with these filevalues (rgb's).

Note that the calculateColor is done for ((nx*ny)+1024-1)/1024 blocks of 1024 threads, this is exactly what is needed for all the rays.

The code for cpu can be seen at the end of this report, in the full code.

```
auto millisec_since_epoch1 =
    duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
for (int k = 0; k < iterations; k++) { //for every frame
    std::string filename = "output" + std::to_string(k) + ".ppm";
    std::ofstream file(filename);
    file << "P3\n" << nx << " " << ny << "\n255\n";

    //list[0] = sphere(vec3(0,100000,-1),99985,vec3(0.1,0.1,0.1));
        //ground - gray
    //list[1] = sphere(vec3(0,-2,-60),7,vec3(0,1,0)); //green
    list[2] = sphere(vec3(-30+2*k,-1,-80),5,vec3(1,0,0)); //red ,
        change of position

    cudaMemcpy(d_list, list, sizeof(sphere) * length_list,
        cudaMemcpyHostToDevice);


    calculateColor<<<((nx*ny)+1024-1)/1024, 1024>>>(d_fileValues,
        nx, ny, lowerleftcorner, horizontal, vertical, origin,
        d_list, length_list);
    cudaDeviceSynchronize();
```

```
    cudaMemcpy(&fileValues, d_fileValues, sizeFileValues,
        cudaMemcpyDeviceToHost);

    for (int j = 0; j < ny; j++) { //This is just to create the file
        for (int i = 0; i < nx; i++) {
            int ir = fileValues[(j * nx + i) * 3];
            int ig = fileValues[(j * nx + i) * 3 + 1];
            int ib = fileValues[(j * nx + i) * 3 + 2];
            file << ir << " " << ig << " " << ib << "\n";
        }
    }

    file.close();
}
```

The function calculateColor() is done for every ray. The integers i and j define which pixel is done in that specific thread. The integers u and v represent the position of the pixel as a percentage (divided by 100) relative to the origin, ranging from 0% to the maximum width and height of the image. This way a ray can be created with origin (0,0,0) as start point with direction (lowerleftcorner + u*horizontal + v*vertical). The direction is actually also the position of the pixel in the plane with respect to the origin. The color is received with the function color(). The rgb values range from 0 to 1, meaning they should be set to a range of 0 to 255 again. Finally the colors are added to the allocated memory that will later on be copied back to the CPU.

Listing 2: global function: calculateColor

```
__global__ void calculateColor(int *fileValues, int nx, int ny, vec3
    lowerleftcorner, vec3 horizontal, vec3 vertical, vec3 origin,
    sphere *list, int length_list) {
int index = blockIdx.x * blockDim.x + threadIdx.x;
int i = index % nx; //columns
int j = index / nx; //rows


float u = float(i) / float(nx);
float v = float(j) / float(ny);
ray r(origin, lowerleftcorner + u*horizontal + v*vertical);
vec3 col = color(r, list, length_list);
int ir = int(255.99*col[0]);
int ig = int(255.99*col[1]);
```

```
    int ib = int(255.99*col[2]);

    int startIndex = index * 3;
    fileValues[startIndex] = ir; //r
    fileValues[startIndex + 1] = ig; //g
    fileValues[startIndex + 2] = ib; //b

}
```

To get the color, every sphere should be checked for a hit with a ray. It's also important that the closest sphere defines the color. As said before, the parameter t used in calculating if a ray hits a sphere, this parameter is also the distance to the circle. So if for example the ray is checked for a hit with a sphere, the t is changed to a lower value than the predefined value of 10000000. If the ray is now checked for a hit with a other sphere and the t turns out lower, meaning the sphere is closer, the new color should be adopted. So note that every ray has to do a calculation for each sphere whether it has hit or not. If no hit happened, the background should be returned, which is a blue shading.

Listing 3: color()

```
__host__ __device__ vec3 color(const ray& r, sphere* list, int
    length_list) {

float closest_t = 10000000; // large value
vec3 closest_color(0, 0, 0);

for (int i = 0; i < length_list; i++) {
    float t = hit_sphere(list[i].getCenter(), list[i].getRadius(),
        r);

    if ( t > 0.0 && t < closest_t) {
        closest_t = t;
        vec3 N = list[i].getColor();
        closest_color = N;
    }
}

// If a hit occurred, return the color
if (closest_t < 10000000) {
    return closest_color;
}
```

```
    //following is the background:
    vec3 unit_direction = unit_vector(r.direction());
    closest_t = 0.5*(unit_direction.y() + 1.0);
    return (1.0-closest_t)*vec3(1.0, 1.0, 1.0) + closest_t*vec3(0.5,
        0.7, 1.0);
}
```

Hit_sphere() function has the radius, center and ray as it's arguments. This is all that is needed to calculate t according to previously explained formula:

$$t^2 \cdot \text{dot}(B, B) + 2t \cdot \text{dot}(B, A - C) + \text{dot}(A - C, A - C) - R^2 = 0$$

This formula is solved with the discriminant equations: Discriminant $= B^2 - 4AC$. If this is bigger then 0, one or two solutions are found. The integer t proportional to the distance is returned. t is calculated like this:

$$t = \frac{-B \pm \sqrt{B^2 - 4AC}}{2A}$$

Note that all functions used in the global function need "__host__" and "__device__" in front of them. This means they can be called from both the CPU and GPU (host and device).

Listing 4: hit_sphere()

```
    __host__ __device__ float hit_sphere(const vec3& center, float
        radius, const ray& r) {
    vec3 oc = r.origin() - center;
    float a = dot(r.direction(), r.direction());
    float b = 2.0 * dot(oc, r.direction());
    float c = dot(oc, oc) - radius*radius;
    float discriminant = b*b - 4*a*c;

    if(discriminant < 0){
        return -1.0;
    }else {
        return(-b -sqrt(discriminant))/(2.0*a);
    }
}
```

4 results

The result consists of 30 pictures with a red and green sphere. Every frame, the red sphere moves a little bit as depicted in figure 3.

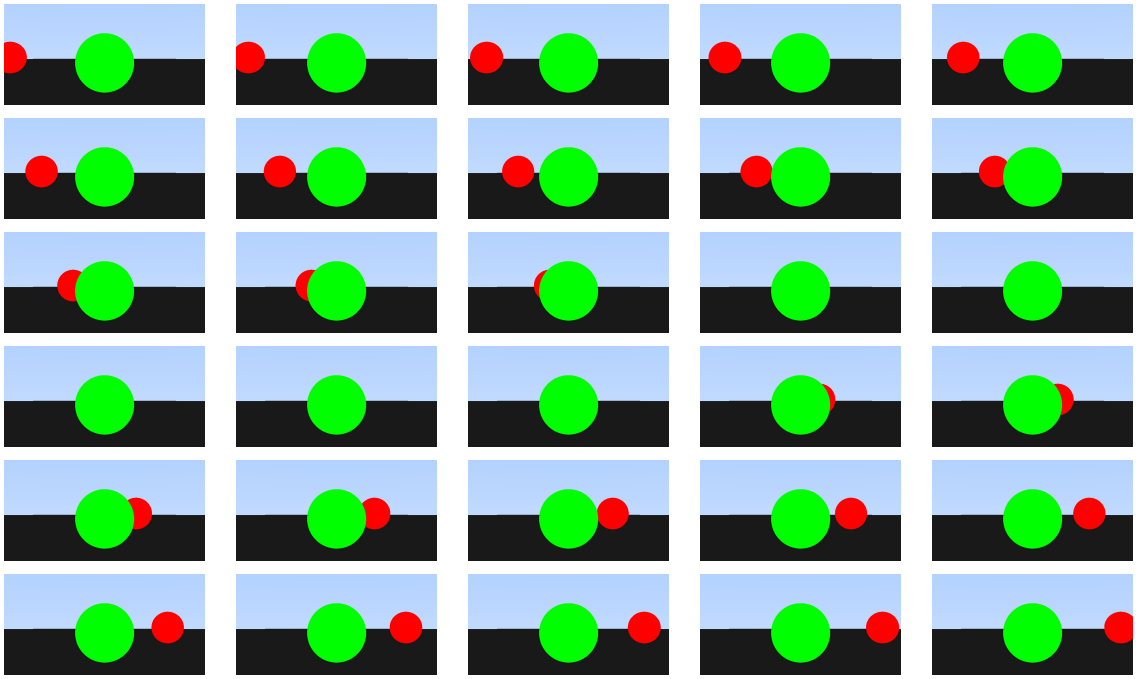


Figure 3: Results moving sphere

The timing results (in microseconds) for the GPU are better then CPU. The GPU is done in 2 seconds while the cpu takes 5! This means a great improvement even though in both timing intervals, file making and writing is also done, which creates an offset in timing.

```
timeDeltaGpu: 2077043
timeDeltaCpu: 5115926
```

Figure 4: Timing results



References

[1] https://github.com/RayTracing/raytracing.github.io/blob/release/books/RayTracingInOneWeekend.html.

[2] https://nl.wikipedia.org/wiki/Raytracing.

[3] https://developer.nvidia.com/blog/accelerated-ray-tracing-cuda/.

All code:

Listing 5: main.cu

```
#include <iostream>
#include <fstream> //voor het schrijven naar file
#include "sphere.h"
#include "float.h"
#include "vec3.h"
#include "ray.h"

#include <chrono>
#include <stdio.h>
#include <time.h>
#include <sys/time.h>
#include <ctime>
using std::chrono::duration_cast;
using std::chrono::milliseconds;
using std::chrono::microseconds;
using std::chrono::seconds;
using std::chrono::system_clock;

time_t start = time(0);

__host__ __device__ float hit_sphere(const vec3& center, float radius,
    const ray& r) {
    vec3 oc = r.origin() - center;
    float a = dot(r.direction(), r.direction());
    float b = 2.0 * dot(oc, r.direction());
    float c = dot(oc, oc) - radius*radius;
    float discriminant = b*b - 4*a*c;

    if(discriminant < 0){
        return -1.0;
    }else {
        return(-b -sqrt(discriminant))/(2.0*a);
    }
}

__host__ __device__ vec3 color(const ray& r, sphere* list, int
    length_list) {

    float closest_t = 10000000; // large value
    vec3 closest_color(0, 0, 0);
```

```
    for (int i = 0; i < length_list; i++) {
        float t = hit_sphere(list[i].getCenter(), list[i].getRadius(),
            r);

        if ( t > 0.0 && t < closest_t) {
            closest_t = t;
            vec3 N = list[i].getColor(); //r.point_at_parameter(t) ,
                deze geeft met gebruik van t, de vector terug die wijst
                naar het oppervlak van de cirkel
            closest_color = N;
        }
    }

    // If a hit occurred, return the color
    if (closest_t < 10000000) {
        return closest_color;
    }

    //following is the background:
    vec3 unit_direction = unit_vector(r.direction());
    closest_t = 0.5*(unit_direction.y() + 1.0);
    return (1.0-closest_t)*vec3(1.0, 1.0, 1.0) + closest_t*vec3(0.5,
        0.7, 1.0);
}


__global__ void calculateColor(int *fileValues, int nx, int ny, vec3
    lowerleftcorner, vec3 horizontal, vec3 vertical, vec3 origin, sphere
    *list, int length_list) {
    int index = blockIdx.x * blockDim.x + threadIdx.x;
    int i = index % nx; //columns
    int j = index / nx; //rows


    float u = float(i) / float(nx);
    float v = float(j) / float(ny);
    ray r(origin, lowerleftcorner + u*horizontal + v*vertical);
    vec3 col = color(r, list, length_list);
    int ir = int(255.99*col[0]);
    int ig = int(255.99*col[1]);
    int ib = int(255.99*col[2]);
```

```
    int startIndex = index * 3;
    fileValues[startIndex] = ir; //r
    fileValues[startIndex + 1] = ig; //g
    fileValues[startIndex + 2] = ib; //b

}

int main() {
    int nx = 800;
    int ny = 400;
    int iterations = 30;

    int fileValues[nx*ny*3]; //alle rgb waarden
    int length_list = 3;
    sphere list[length_list];
    list[0] = sphere(vec3(0,100000,-1),99985,vec3(0.1,0.1,0.1)); //ground
    list[1] = sphere(vec3(0,-2,-60),7,vec3(0,1,0)); //green
    list[2] = sphere(vec3(20,-1,-80),5,vec3(1,0,0)); //blue

    //window I look at:
    vec3 lowerleftcorner(-2.0,-1.0,-5);
    vec3 horizontal(4.0,0.0,0.0);
    vec3 vertical(0.0,2.0,0.0);
    vec3 origin(0.0,0.0,0.0);

    int * d_fileValues;
    int sizeFileValues = sizeof(fileValues);
    sphere *d_list;

    cudaMalloc((void **)&d_fileValues, sizeFileValues);
    cudaMalloc((void**)&d_list, sizeof(sphere) * length_list);

    auto millisec_since_epoch1 =
        duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
    for (int k = 0; k < iterations; k++) {
        std::string filename = "output" + std::to_string(k) + ".ppm";
        std::ofstream file(filename);
        file << "P3\n" << nx << " " << ny << "\n255\n";

        list[0] = sphere(vec3(0,100000,-1),99985,vec3(0.1,0.1,0.1));
            //ground
        list[1] = sphere(vec3(0,-2,-60),7,vec3(0,1,0)); //green
        list[2] = sphere(vec3(-30+2*k,-1,-80),5,vec3(1,0,0)); //blue
```

```
    cudaMemcpy(d_list, list, sizeof(sphere) * length_list,
        cudaMemcpyHostToDevice);

    calculateColor<<<((nx*ny)+1024-1)/1024, 1024>>>(d_fileValues,
        nx, ny, lowerleftcorner, horizontal, vertical, origin,
        d_list, length_list);
    cudaDeviceSynchronize();

    cudaMemcpy(&fileValues, d_fileValues, sizeFileValues,
        cudaMemcpyDeviceToHost);//ging ik in array zetten filevalues

    for (int j = 0; j < ny; j++) {
        for (int i = 0; i < nx; i++) {
            int ir = fileValues[(j * nx + i) * 3];
            int ig = fileValues[(j * nx + i) * 3 + 1];
            int ib = fileValues[(j * nx + i) * 3 + 2];
            file << ir << " " << ig << " " << ib << "\n";
        }
    }

    file.close();
}
auto millisec_since_epoch2 =
    duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
printf("timeDeltaGpu: ");
printf("%ld\n", millisec_since_epoch2 - millisec_since_epoch1);


auto millisec_since_epoch3 =
    duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
for (int k = 0; k < iterations; k++) {
    std::string filename = "output" + std::to_string(k) + ".ppm";
    std::ofstream file(filename);
    file << "P3\n" << nx << " " << ny << "\n255\n";

    list[0] = sphere(vec3(0,-100000,-1),99985,vec3(0.1,0.1,0.1));
        //ground
    list[1] = sphere(vec3(0,-2,-60),7,vec3(0,1,0)); //green
    list[2] = sphere(vec3(-30+2*k,-1,-80),5,vec3(1,0,0)); //blue

    /*code zonder cuda:*/
    int index = 0;
```

```
        for (int j = ny-1; j >= 0; j--) {
            for (int i = 0; i < nx; i++) {
                float u = float(i) / float(nx);
                float v = float(j) / float(ny);
                ray r(origin, lowerleftcorner + u*horizontal +
                    v*vertical);
                vec3 col = color(r, list,length_list);
                int ir = int(255.99*col[0]);
                int ig = int(255.99*col[1]);
                int ib = int(255.99*col[2]);


                fileValues[index++] = ir;
                fileValues[index++] = ig;
                fileValues[index++] = ib;
            }
        }

        for (int j = 0; j < ny; j++) {
            for (int i = 0; i < nx; i++) {
                int ir = fileValues[(j * nx + i) * 3];
                int ig = fileValues[(j * nx + i) * 3 + 1];
                int ib = fileValues[(j * nx + i) * 3 + 2];
                file << ir << " " << ig << " " << ib << "\n";
            }
        }

        file.close();
    }
    auto millisec_since_epoch4 =
        duration_cast<microseconds>(system_clock::now().time_since_epoch()).count();
    printf("timeDeltaCpu: ");
    printf("%ld\n", millisec_since_epoch4 - millisec_since_epoch3);


    cudaFree(d_fileValues);
    cudaFree(d_list);
}
```

Listing 6: vec3.h

```
#ifndef VEC3H
#define VEC3H

#include <math.h>
#include <stdlib.h>
#include <iostream>

class vec3 {


public:
    __host__ __device__ vec3() {}
    __host__ __device__ vec3(float e0, float e1, float e2) { e[0] = e0;
       e[1] = e1; e[2] = e2; }
    __host__ __device__ inline float x() const { return e[0]; }
    __host__ __device__ inline float y() const { return e[1]; }
    __host__ __device__ inline float z() const { return e[2]; }
    __host__ __device__ inline float r() const { return e[0]; }
    __host__ __device__ inline float g() const { return e[1]; }
    __host__ __device__ inline float b() const { return e[2]; }

    __host__ __device__ inline const vec3& operator+() const { return
       *this; }
    __host__ __device__ inline vec3 operator-() const { return
       vec3(-e[0], -e[1], -e[2]); }
    __host__ __device__ inline float operator[](int i) const { return
       e[i]; }
    __host__ __device__ inline float& operator[](int i) { return e[i]; };

    __host__ __device__ inline vec3& operator+=(const vec3 &v2);
    __host__ __device__ inline vec3& operator-=(const vec3 &v2);
    __host__ __device__ inline vec3& operator*=(const vec3 &v2);
    __host__ __device__ inline vec3& operator/=(const vec3 &v2);
    __host__ __device__ inline vec3& operator*=(const float t);
    __host__ __device__ inline vec3& operator/=(const float t);

    __host__ __device__ inline float length() const { return
       sqrt(e[0]*e[0] + e[1]*e[1] + e[2]*e[2]); }
    __host__ __device__ inline float squared_length() const { return
       e[0]*e[0] + e[1]*e[1] + e[2]*e[2]; }
    __host__ __device__ inline void make_unit_vector();
```

```
    float e[3];
};



inline std::istream& operator>>(std::istream &is, vec3 &t) {
    is >> t.e[0] >> t.e[1] >> t.e[2];
    return is;
}

inline std::ostream& operator<<(std::ostream &os, const vec3 &t) {
    os << t.e[0] << " " << t.e[1] << " " << t.e[2];
    return os;
}

__host__ __device__ inline void vec3::make_unit_vector() {
    float k = 1.0 / sqrt(e[0]*e[0] + e[1]*e[1] + e[2]*e[2]);
    e[0] *= k; e[1] *= k; e[2] *= k;
}

__host__ __device__ inline vec3 operator+(const vec3 &v1, const vec3
    &v2) {
    return vec3(v1.e[0] + v2.e[0], v1.e[1] + v2.e[1], v1.e[2] + v2.e[2]);
}

__host__ __device__ inline vec3 operator-(const vec3 &v1, const vec3
    &v2) {
    return vec3(v1.e[0] - v2.e[0], v1.e[1] - v2.e[1], v1.e[2] - v2.e[2]);
}

__host__ __device__ inline vec3 operator*(const vec3 &v1, const vec3
    &v2) {
    return vec3(v1.e[0] * v2.e[0], v1.e[1] * v2.e[1], v1.e[2] * v2.e[2]);
}

__host__ __device__ inline vec3 operator/(const vec3 &v1, const vec3
    &v2) {
    return vec3(v1.e[0] / v2.e[0], v1.e[1] / v2.e[1], v1.e[2] / v2.e[2]);
}

__host__ __device__ inline vec3 operator*(float t, const vec3 &v) {
    return vec3(t*v.e[0], t*v.e[1], t*v.e[2]);
```

```
}

__host__ __device__ inline vec3 operator/(vec3 v, float t) {
    return vec3(v.e[0]/t, v.e[1]/t, v.e[2]/t);
}

__host__ __device__ inline vec3 operator*(const vec3 &v, float t) {
    return vec3(t*v.e[0], t*v.e[1], t*v.e[2]);
}

__host__ __device__ inline float dot(const vec3 &v1, const vec3 &v2) {
    return v1.e[0] *v2.e[0] + v1.e[1] *v2.e[1] + v1.e[2] *v2.e[2];
}

__host__ __device__ inline vec3 cross(const vec3 &v1, const vec3 &v2) {
    return vec3( (v1.e[1]*v2.e[2] - v1.e[2]*v2.e[1]),
                (-(v1.e[0]*v2.e[2] - v1.e[2]*v2.e[0])),
                (v1.e[0]*v2.e[1] - v1.e[1]*v2.e[0]));
}


__host__ __device__ inline vec3& vec3::operator+=(const vec3 &v){
    e[0]  += v.e[0];
    e[1]  += v.e[1];
    e[2]  += v.e[2];
    return *this;
}

__host__ __device__ inline vec3& vec3::operator*=(const vec3 &v){
    e[0]  *= v.e[0];
    e[1]  *= v.e[1];
    e[2]  *= v.e[2];
    return *this;
}

__host__ __device__ inline vec3& vec3::operator/=(const vec3 &v){
    e[0]  /= v.e[0];
    e[1]  /= v.e[1];
    e[2]  /= v.e[2];
    return *this;
}

__host__ __device__ inline vec3& vec3::operator-=(const vec3& v) {
```

```
    e[0]  -= v.e[0];
    e[1]  -= v.e[1];
    e[2]  -= v.e[2];
    return *this;
}

__host__ __device__ inline vec3& vec3::operator*=(const float t) {
    e[0]  *= t;
    e[1]  *= t;
    e[2]  *= t;
    return *this;
}

__host__ __device__ inline vec3& vec3::operator/=(const float t) {
    float k = 1.0/t;

    e[0]  *= k;
    e[1]  *= k;
    e[2]  *= k;
    return *this;
}

__host__ __device__ inline vec3 unit_vector(vec3 v) {
    return v / v.length();
}

#endif
```

Listing 7: sphere.h

```
#ifndef SPHEREH
#define SPHEREH

#include <curand_kernel.h>
#include "vec3.h"

class sphere {
    public:
        __host__ __device__ sphere() {}
        __host__ __device__ sphere(vec3 cen, float r, vec3 c) :
            center(cen), radius(r), color(c) {}
        // __host__ __device__ virtual bool hit(const ray& r, float
            tmin, float tmax, hit_record& rec) const;
            // Getters
        __host__ __device__ vec3 getCenter() const { return center; }
        __host__ __device__ float getRadius() const { return radius; }
        __host__ __device__ vec3 getColor() const { return color; }

    private:
        vec3 center;
        float radius;
        vec3 color;
};


#endif
```

Listing 8: ray.h

```
#ifndef RAYH
#define RAYH
#include "vec3.h"

class ray
{
    public:
        __host__ __device__ ray() {}
        __host__ __device__ ray(const vec3& a, const vec3& b) { A = a; B
            = b; }
        __host__ __device__ vec3 origin() const   { return A; }
        __host__ __device__ vec3 direction() const { return B; }
        __host__ __device__ vec3 point_at_parameter(float t) const {
            return A + t*B; }

        vec3 A; //origin
        vec3 B; //direction
};

#endif
```